

Coding & Solution

Date	01 JULY 2026
Team ID	XXXXX
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

I

Solution Summary

Field	Details
Repository Link / URL	Github link : https://github.com/Abhilash-18-tech/Credit-Card-Approval-Prediction Live url : https://credit-card-approval-2her.onrender.com/
Programming Language(s)	PYTHON,HTML,CSS,JAVA SCRIPT
Framework(s) Used	FLASK
Key Features Implemented	1. Automated Assets Build Pipeline (build_assets.py): A single script executing end-to-end steps (downloading, target engineering, preprocessing fitting, and production model training). 2. Repayment Target Engineering (src/data_pipeline.py): Formulates binary risk targets (TARGET=1 default risk, TARGET=0 good credit history) by aggregating monthly payment statuses. 3. Serialized Preprocessing Pipelines (src/data_preprocessing.py): Bundles scaling, imputing, and one-hot encoding pipelines, allowing portability between training and inference. 4. Interactive Dashboard UI (app.py &

	templates/index.html): Modern panel navigation featuring a submission form, real-time prediction (risk level, approval score), metric plots, and a gauge meter.
Pending / Incomplete Features	1. Centralized Configuration File: Relies on hardcoded values (URLs, columns, model params) within source scripts; moving these to a config file (e.g. config.yaml) would make updates easier. 2. Unit / Integration Tests: Formal test suites (e.g. pytest or unittest) are not currently set up to verify pipeline transformations or API endpoints. 3. Exported Application Monitoring: Logs print directly to stdout/stderr; setting up structured logger exports for production monitoring is missing.
Setup / Run Instructions	1. Activate Environment: Set up virtual environment (python -m venv venv) and activate (.venv\Scripts\Activate.ps1). 2. Install Packages: Install dependencies: pip install -r requirements.txt. 3. Build Assets: Run the full assets pipeline script: python build_assets.py. 4. Start Web Server: Launch Flask app: python app.py and open http://127.0.0.1:5000 in your browser.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	YES
2	Meaningful variable and function names are used	YES
3	Code includes comments / documentation where necessary	YES
4	Error handling is implemented for critical operations	YES
5	The application runs without critical errors	YES
6	Code is committed to a version control repository	YES

Additional Notes / Comments:

- Imbalance & Leakage Management: Handling class weights (class_weight='balanced') in tree classifiers and encapsulating feature operations in a serialized ColumnTransformer (fitting only on the training split) shows high-quality machine learning practices.
- Error Handling: Implements try-except blocks for network I/O during data fetch in

src/utlis.py, safe startup checks in

app.py, and defensive type-casting on web form fields in

app.py.

- Git Version Control: Active .git tracker present at the workspace root, properly configured with a .gitignore to exclude large CSVs and serialized model binaries.